

# Task API CRUD Check Results

**PASS: Both full CRUD cycles completed.**

## Swagger UI — Try it out

| Action                               | Result                             | Status |
|--------------------------------------|------------------------------------|--------|
| POST /tasks — create “Buy milk”      | Task id 4 created                  | 201    |
| GET /tasks — list tasks              | New task appeared in the list      | 200    |
| PUT /tasks/4 — update title and done | “Buy milk and bread”, done=true    | 200    |
| DELETE /tasks/4                      | Task deleted; response had no body | 204    |

## curl -i — full CRUD cycle

| Request                     | Status         | Result                      |
|-----------------------------|----------------|-----------------------------|
| POST /tasks                 | 201 Created    | Created id 4                |
| GET /tasks                  | 200 OK         | List included id 4          |
| PUT /tasks/4                | 200 OK         | Updated title and done=true |
| DELETE /tasks/4             | 204 No Content | Empty response body         |
| GET /tasks/4 after deletion | 404 Not Found  | Task 4 not found            |

```
HTTP/1.1 201 Created
date: Fri, 25 Sep 2026 19:49:08 GMT
server: uvicorn
content-length: 49
content-type: application/json

{"id":4,"title":"Buy milk via curl","done":false}
```

## Terminal input and output by stage

Stages 0–3 were run from their original local commit snapshots on temporary servers. Stage 4 used the final API. Stage 6 was checked from a fresh clone of the public repository. The response blocks show the status, content type, and body from the actual terminal output; date and content length headers are omitted for readability.

### ▼ Stage 0 — Hello server

```
$ curl.exe -i -s http://127.0.0.1:8100/
HTTP/1.1 200 OK
content-type: application/json

{"message":"Hello World"}
```

### ▼ Stage 1 — Root and health endpoints

```
$ curl.exe -i -s http://127.0.0.1:8101/Tasks
HTTP/1.1 200 OK
content-type: application/json

{"name":"Task API","version":"1.0","endpoints":["/Tasks"]}

$ curl.exe -i -s http://127.0.0.1:8101/health
HTTP/1.1 200 OK
content-type: application/json

{"status":"ok"}
```

### ▼ Stage 2 — List and read tasks

```
$ curl.exe -i -s http://127.0.0.1:8102/tasks
HTTP/1.1 200 OK
content-type: application/json

[{"id":1,"title":"Learn FastAPI","done":false}, {"id":2,"title":"Build REST API","done":true}, {"id":3,"title":"Practice Python","done":false}]

$ curl.exe -i -s http://127.0.0.1:8102/tasks/1
HTTP/1.1 200 OK
content-type: application/json

{"error":"Task 1 not found"}
```

The Stage 2 read endpoint had a bug: it returned an error message with status 200 for an existing task. Stage 4 fixed it.

### ▼ Stage 3 — Create with validation

```
$ curl.exe -i -s -X POST http://127.0.0.1:8103/tasks -H 'Content-Type: application/json' --data-raw '{"title":"Buy milk"}'
HTTP/1.1 201 Created
content-type: application/json

{"id":4,"title":"Buy milk","done":false}

$ curl.exe -i -s -X POST http://127.0.0.1:8103/tasks -H 'Content-Type: application/json' --data-raw '{"title":""}'
HTTP/1.1 400 Bad Request
content-type: application/json

{"detail":"Title is required and cannot be empty"}
```

## ▼ Stage 4 — Complete CRUD

```
$ curl.exe -i -s -X POST http://127.0.0.1:8001/tasks -H 'Content-Type: application/json' --data-raw '{"title":"Buy milk"}'
HTTP/1.1 201 Created
content-type: application/json

{"id":4,"title":"Buy milk","done":false}

$ curl.exe -i -s http://127.0.0.1:8001/tasks
HTTP/1.1 200 OK
content-type: application/json

[{"id":1,"title":"Learn FastAPI","done":false},{ "id":2,"title":"Build REST API","done":true},{ "id":3,"title":"Practice Python","done":false},{ "id":4,"title":"Buy milk","done":false}]

$ curl.exe -i -s -X PUT http://127.0.0.1:8001/tasks/4 -H 'Content-Type: application/json' --data-raw '{"title":"Buy milk and bread","done":true}'
HTTP/1.1 200 OK
content-type: application/json

{"id":4,"title":"Buy milk and bread","done":true}

$ curl.exe -i -s -X DELETE http://127.0.0.1:8001/tasks/4
HTTP/1.1 204 No Content

$ curl.exe -i -s http://127.0.0.1:8001/tasks/4
HTTP/1.1 404 Not Found
content-type: application/json

{"detail":"Task 4 not found"}
```

## ▼ Stage 5 — Swagger UI

```
$ curl.exe -i -s http://127.0.0.1:8001/docs
HTTP/1.1 200 OK
content-type: text/html; charset=utf-8

<title>FastAPI - Swagger UI</title>

$s = Invoke-RestMethod 'http://127.0.0.1:8001/openapi.json'; [pscustomobject]@{POST_body=
[bool]$s.paths['/tasks'].post.requestBody; PUT_body=[bool]$s.paths['/tasks/{id}'].put.requestBody; DELETE_body=
[bool]$s.paths['/tasks/{id}'].delete.requestBody} | Format-List
POST_body    : True
PUT_body     : True
DELETE_body  : False
```

The HTML response above shows its title. The full CRUD cycle was also completed in Swagger UI using “Try it out” as shown in the table.

## ▼ Stage 6 — Public GitHub repository

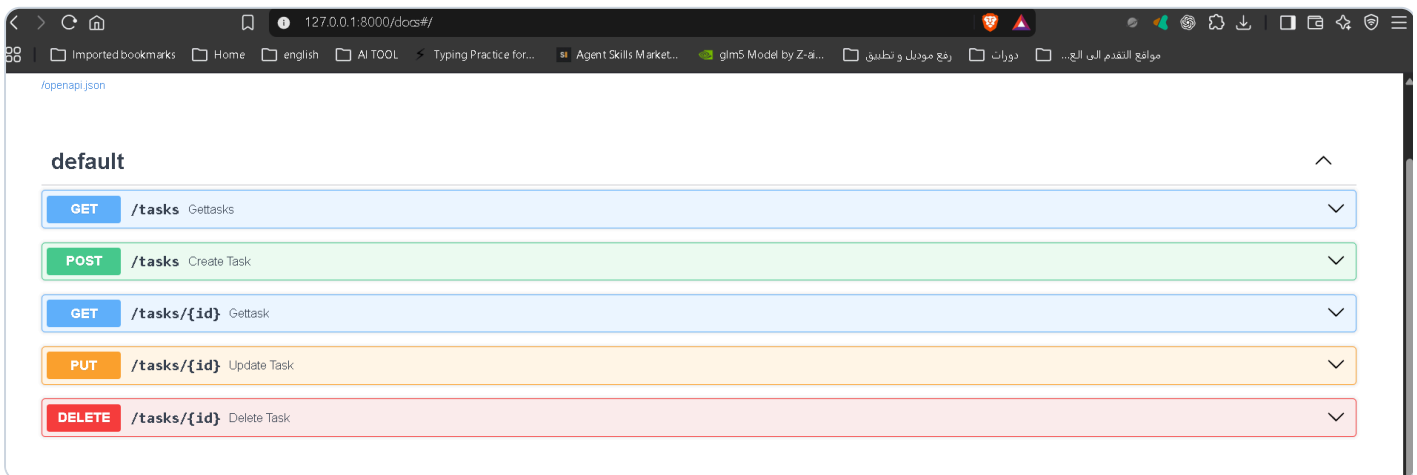
```
$ git clone --depth 10 https://github.com/mike-elio/crud.git .stage6-check
Cloning into '.stage6-check'...

$ git log --oneline -8
012afa4 Stage 6: publish with README
daabbb8 Stage 5: Swagger UI
c2207fa Stage 4: complete task CRUD API
78280be Stage 3: create with validation
eb29759 Stage 2: list and read tasks
6f7eb0f Stage 1: root and health endpoints
d8df7c3 Stage 0: hello server
ea13884 Initial commit
```

[Open the public repository](#). Its README documents installation, one run command, endpoints, and a captured curl response.

## Swagger UI screenshots

These screenshots show additional “Try it out” examples on port 8000. Some examples use different task IDs from the verified CRUD cycle above.



All five task endpoints appear in Swagger UI.

POST /tasks Create Task

Parameters

No parameters

Request body required

application/json

Edit Value | Schema

```
{
  "title": "buy milk "
}
```

Execute

POST /tasks has a JSON request body editor.

```
}'
```

Request URL

http://127.0.0.1:8000/tasks

Server response

| Code | Details                                                                                                                                                                                                                                         |
|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 201  | <div><div>Response body</div><pre>{   "id": 4,   "title": "buy milk ",   "done": false }</pre><div>Response headers</div><pre>content-length: 41 content-type: application/json date: Fri, 25 Sep 2026 20:16:53 GMT server: uvicorn</pre></div> |

Responses

| Code | Description                                                                                                                                                                                |
|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 201  | <div><div>Successful Response</div><div>Media type</div><div>application/json</div><div>Controls Accept header.</div><div><div>Example Value   Schema</div><div>"string"</div></div></div> |

POST /tasks created a task with status 201.

**Responses**

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/tasks' \
  -H 'accept: */*'
```

Request URL

```
http://127.0.0.1:8000/tasks
```

Server response

| Code | Details                                                                                                                                                                                                                                                                |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 200  | <p>Response body</p> <pre>[   {     "id": 1,     "title": "Learn FastAPI",     "done": false   },   {     "id": 2,     "title": "Build REST API",     "done": true   },   {     "id": 3,     "title": "Practice Python",     "done": false   } ]</pre> <p>Download</p> |

GET /tasks returned the in-memory list with status 200.

**GET** /tasks/{id} Gettask

[Try it out](#)

| Name                                      | Description                     |
|-------------------------------------------|---------------------------------|
| <b>id</b> * required<br>integer<br>(path) | <input type="text" value="id"/> |

**Responses**

| Code | Description         | Links    |
|------|---------------------|----------|
| 200  | Successful Response | No links |

Media type:

Controls: [Accept header](#)

GET /tasks/{id} exposes the required path parameter.

Execute

Responses

Curl

```
curl -X 'GET' \
'http://127.0.0.1:8000/tasks/2' \
-H 'accept: */*'
```

Request URL

```
http://127.0.0.1:8000/tasks/2
```

Server response

| Code | Details                                                                                                                                                                                                                                                                              |
|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 200  | <div><div>Response body</div><div><pre>{   "id": 2,   "title": "Build REST API",   "done": true }</pre></div><div><div>Response headers</div><div><pre>content-length: 45 content-type: application/json date: Fri, 25 Sep 2026 20:17:52 GMT server: uvicorn</pre></div></div></div> |

Responses

| Code | Description |
|------|-------------|
|------|-------------|

GET /tasks/2 returned an individual task with status 200.

Responses

Curl

```
curl -X 'PUT' \
'http://127.0.0.1:8000/tasks/2' \
-H 'accept: */*' \
-H 'Content-Type: application/json' \
-d '{
  "title": "cool",
  "done": true
}'
```

Request URL

```
http://127.0.0.1:8000/tasks/2
```

Server response

| Code | Details                                                                                                                                                                                                                                                                    |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 200  | <div><div>Response body</div><div><pre>{   "id": 2,   "title": "cool",   "done": true }</pre></div><div><div>Response headers</div><div><pre>content-length: 35 content-type: application/json date: Fri, 25 Sep 2026 20:18:46 GMT server: uvicorn</pre></div></div></div> |

Responses

| Code | Description |
|------|-------------|
|------|-------------|

PUT /tasks/2 updated title and done with status 200.

#### Curl

```
curl -X 'DELETE' \
'http://127.0.0.1:8000/tasks/3' \
-H 'accept: */*'
```

#### Request URL

`http://127.0.0.1:8000/tasks/3`

#### Server response

| Code | Details |
|------|---------|
|------|---------|

204

#### Response headers

```
date: Fri, 25 Sep 2026 20:19:23 GMT
server: uvicorn
```

#### Responses

| Code | Description |
|------|-------------|
|------|-------------|

|     |                     |
|-----|---------------------|
| 204 | Successful Response |
|-----|---------------------|

|     |                  |
|-----|------------------|
| 422 | Validation Error |
|-----|------------------|

Media type

**application/json** ▼

**Example Value** | Schema

DELETE /tasks/3 returned 204 with no response body.